EvoHash

LLM-guided
Evolutionary Discovery of
Collision Attacks on
Perceptual Hash Functions

Kirill Frolov
Daniil Koblov
Amir Sadreev
Grigorii Gulii
Yakupova Valeria

601 RUSKOLKEK

Perceptual Hash Functions

map visually similar images to similar or identical hash values

Used in:

- content moderation
- copyright protection
- detection of illegal material

Examples:

- pHash
- PDQ
- NeuralHash
- PhotoDNA

Problem

PHFs must satisfy two conflicting properties:

- robustness to small image transformations
- ability to distinguish different images

Hash collisions is when two distinct pieces of data in a hash table share the same hash value.

Collisions enable:

- bypassing moderation systems
- adversarial content injection
- false positives in detection

Project

Project aims to design and implement EvoHash, a framework that utilizes the GigaEvo evolutionary programming system to automatically discover and optimize complex, multi-scale collision attacks.

Objective:

Develop an automated pipeline that evolves attack code to generate target hash collisions for four widely deployed PHF.

PHFs

pHash (Perceptual Hash)

Converts image to grayscale
Applies DCT and keeps low-frequency coefficients
Binarizes coefficients into a hash
Captures global image structure

Sensitive to structured / frequency-domain attacks

PDQ

Resizes and normalizes the image
Applies DCT-like transform
Generates a binary hash optimized for Hamming distance
Robust to resizing, compression, minor edits

Designed for large-scale matching

PHFs

NeuralHash

Processes image with a neural network
Extracts learned feature representation
Maps features to a compact hash
Robust to complex transformations

Less interpretable than classical methods

PhotoDNA

Not public architecture from Microsoft

Black-box attacks on perceptual hashing systems

1. **Random Noise** — i.i.d. Gaussian perturbations, keep best sample. Minimal robustness baseline.

2. **Sign-SGD family** — estimate gradient via random sampling, update by sign(grad):
   a. NES: antithetic pairs (f(x+σu) − f(x−σu)) — balanced, low variance
   b. SimBa: coordinate-wise updates in pixel/DCT space
   c. ZO-SignSGD: one-sided finite difference (f(x+μu) − f(x)) — cheaper per query
   d. ZO-SignSGD v2: central difference + momentum accumulation — smoother convergence

3. **Frequency-domain attacks** — exploit that pHash/PDQ operate in DCT space:
   a. Prokos: perturbs only low-to-mid DCT frequencies with momentum
   b. AtkScopes: multi-resolution batched probing in DCT domain

4. **Analytical DCT** — directly manipulates DCT coefficients to flip target hash bits. Exact for pHash/PDQ, inapplicable to neural hashes.

Black-box attacks on perceptual hashing systems

1. **HSJA (HopSkipJump)** — decision-based boundary walk. Starts from a known collision, iteratively steps toward source along the decision boundary, minimizing L2 while preserving collision. Requires an initial adversarial example.

2. **HSJA v2** — self-initializing HSJA: finds starting collision automatically (noise → ZO-SignSGD fallback), then boundary walk with adaptive step size.

3. **Two-phase hybrids** — Phase 1: Sign-SGD attack finds collision fast (ignoring L2). Phase 2: HSJA refines it, walking the boundary back toward source to minimize distortion.
   a. NES + HSJA, SimBa + HSJA, ZO-SignSGD + HSJA — conservative Phase 1 (σ=4, lr=2, 80 iter) with mid-step source shortcuts
   b. NES + JSHA, SimBa + JSHA, ZO-SignSGD + JSHA — aggressive Phase 1 (σ=80, lr=8, 800 iter), clean handoff to HSJA refinement (25 iter)

Evaluation methods

1. Success

- **ASR (Attack Success Rate)**: fraction of image pairs that achieve a hash collision.

2. Distortion

- **Mean L2**: normalized pixel-level perturbation magnitude.
- **LPIPS**: perceptual similarity metric aligned with human vision.

3. Efficiency

- **Efficiency**: ASR / (mean L2 + coef * mean LPIPS), balancing success and distortion.

4. Practical cost

- **Queries**: number of hash evaluations (black-box cost).
- **Time**: average runtime per attack.

5. Generalization

- **Transferability**: success rate when attacks transfer across different hashing models.

GigaEvo

Enables automated search for PHF collision attacks

Runs the evolutionary loop (generations, selection, mutation)

Each candidate is a Program object (code+metrics+lineage)

Redis-backed storage

Parallel execution: EvolutionEngine + DagRunner

Pipeline of GigaEvo

1. **ValidateCodeStage:** verify code is syntactically valid and passes basic safety checks.
2. **CallProgramFunction:** execute the program's `entrypoint(context)`.
3. **CallValidatorFunction:** run external `validate(context, payload)` on the produced output.
4. **FetchMetrics:** extract the `metrics` dictionary from the validator output.
5. **FetchArtifact:** extract an auxiliary artifact (if the validator returns one).
6. **InsightsStage**: ask the LLM to analyze the current program.
7. **Collector + Lineage Stages:** collect relatives (parents/children) and run LLM analysis of the parent → child transition.
8. **MutationContextStage:** build a unified mutation context for the next generation.
9. **EnsureMetricsStage:** normalize metrics into a strict schema and write them back into the `Program`.

Experimental Setup

Models:

- GPT-OSS 120B
- GLM-4.7 Flash
- **Qwen 3.5 122B-A10B**

Dataset:

- 100 Random images pairs from ImageNet Val

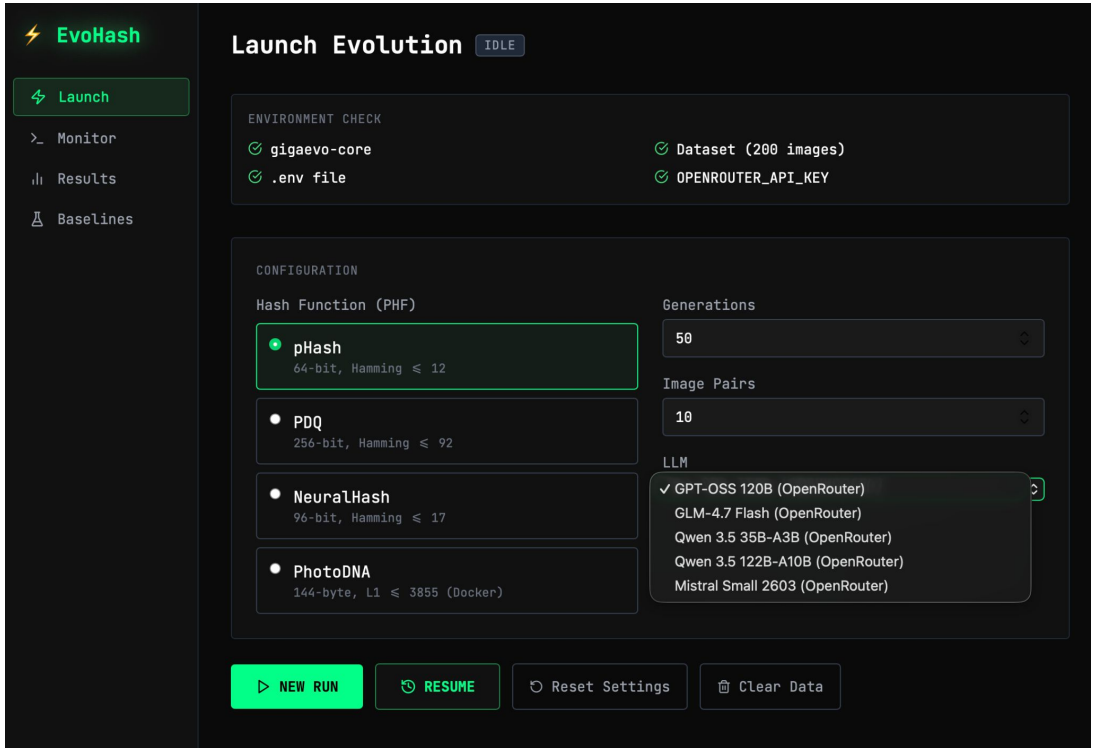
Experiment:

- 20 generations of 10 pairs each

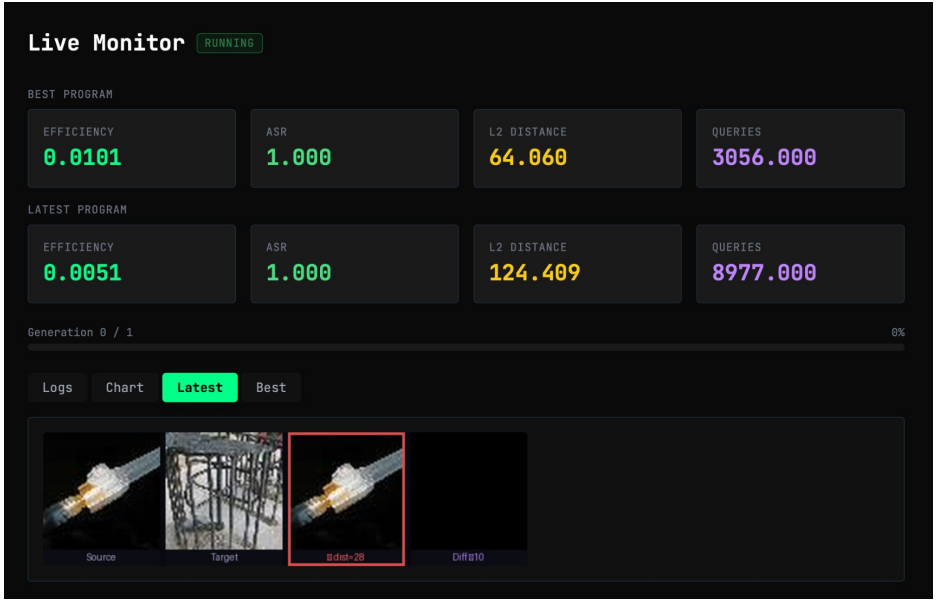
OpenRouter

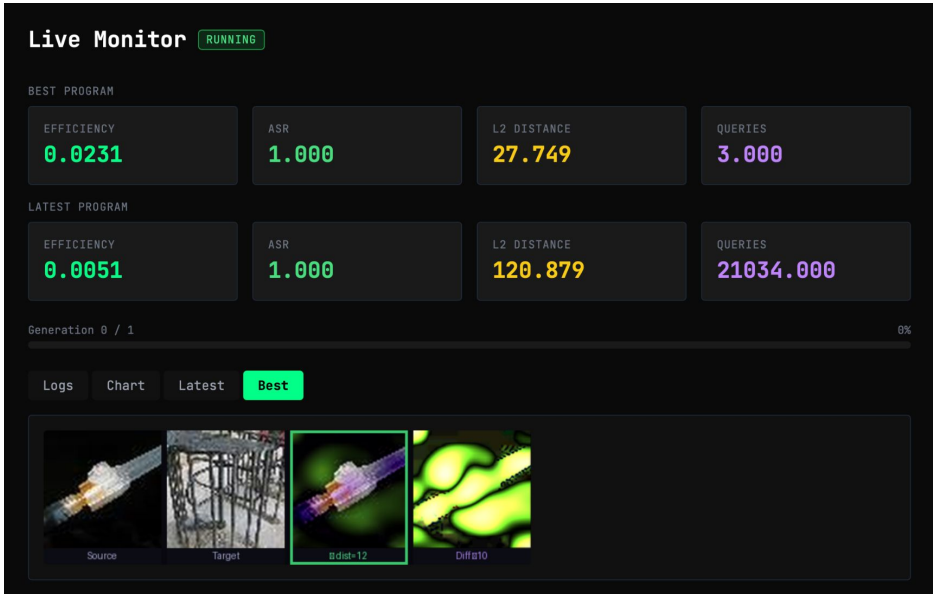
Samples from dataset

Interface for launching experiments

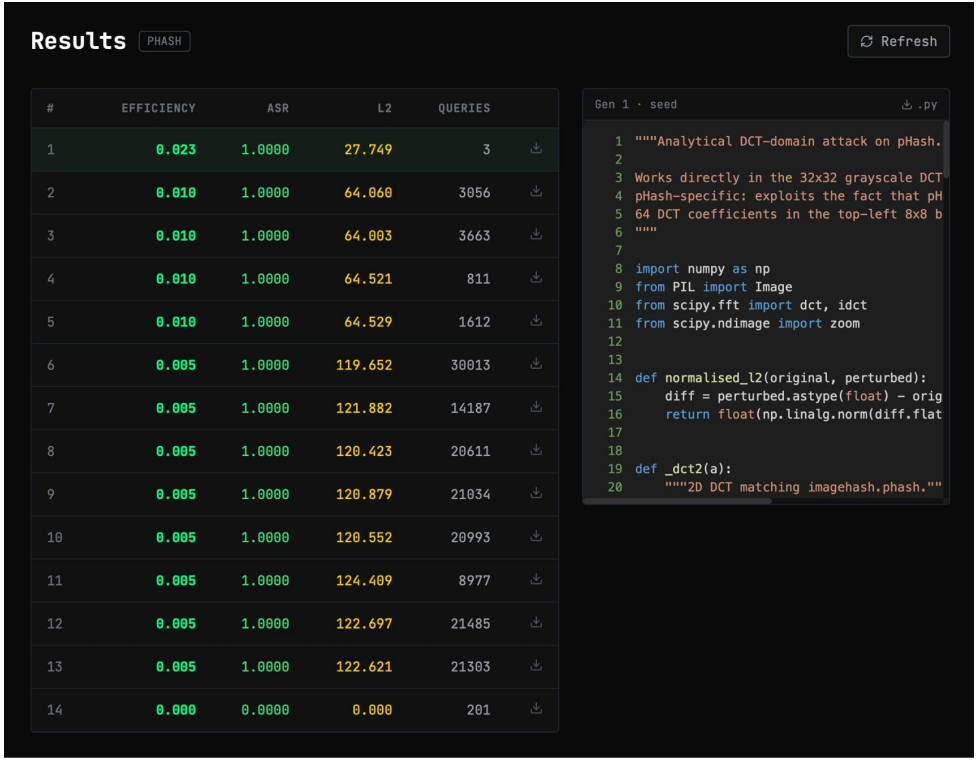


Interface for launching experiments





Interface for launching experiments



Result of evolution for pHash

- Analytical DCT has the best quality among all baseline attacks (and use only 3 queries):

  eff = 0.022, L2 = 32.9, LPIPS = 0.268

- During evolution, the analytical attack mutated over 5 generations (crossed with the NES+HSJA hybrid). This has significantly improved the quality (and use only 2 queries):

  eff = 0.032, L2 = 22.1, LPIPS = 0.183

Samples for final pHash attack

Deeper analysis of pHash attack

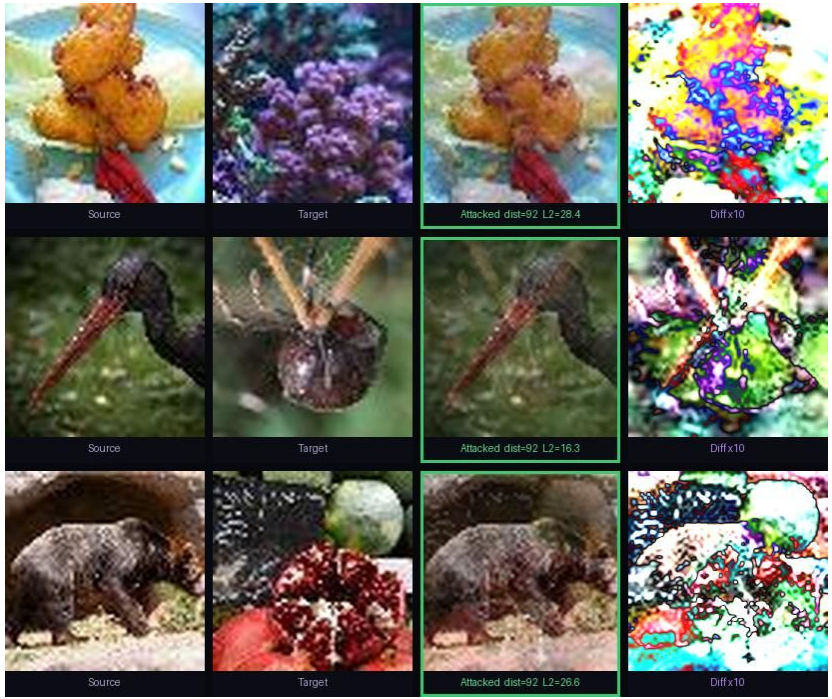
Initial attack

- Fixed margin
- Arbitrary bit processing order
- Only one (analytical) phase of algorithm

Final attack

- Adaptive margin
- Sorting the bits and correcting the worst
- Second phase of boundary walk

Results of evolution for PDQ



- The best baseline for PDQ is NES+HSJA attack (3025 queries):

  eff = 0.0225, L2 = 30, LPIPS = 0.289

- On the 2nd generation, LLM invented an analytical DCT attack. By the 7th generation, this attack was crossed with HSJA-based attacks and the best quality was obtained (requires only 119 queries):

  eff = 0.0321, L2 = 23.9, LPIPS = 0.146

Deeper analysis of PDQ attack

Initial attack

- Long HSJA phase (15 iterations)
- Black-box NES approach in phase 2

Final attack

- Short HSJA phase (5 iterations)
- Analytical DCT approach in phase 2
- Different techniques for adaptation to image, early termination, bit selection

Attack Transferability

	ASR/L2			
Attack \ Target	**PDQ**	**pHash**	**Neural Hash**	**Photo DNA**
PDQ best	🏠 1.00 / 30.5	1.00 / 56.8	1.00 / 71.2	1.00 / 54.9
pHash best	1.00 / 31.0	🏠 1.00 / 22.1	1.00 / 71.1	1.00 / 54.8

Attacks evolved for one PHF tested on all four without retraining

Conclusion

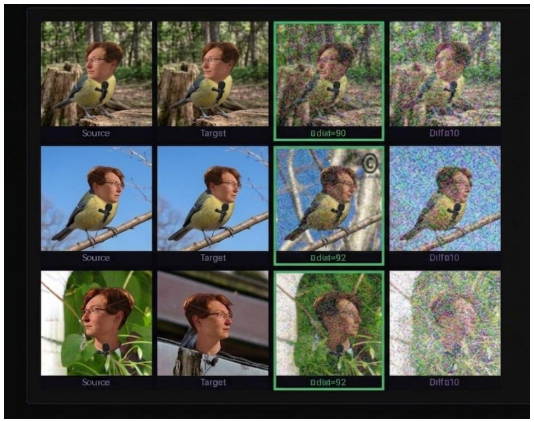
- LLMs can successfully improve existing hash attacks using evolutionary principles in a fairly short time (<10 iterations of evolution)

- High-quality baselines with hyperparameter tuning capabilities/space for implementing improvements are critical to the success of evolution.

- Attacks on NeuralHash/PhotoDNA require much more resources, and perhaps a smarter model for evolution.

Hash project memes

ОПАААААААААААА



У МЕНЯ ТОЖЕ ЕСТЬ ПТИЧКИ

И В ОТЛИЧИЕ ОТ ТВОИХ У НИХ ЕСТЬ МИКРОФОНЫ-ПЕТЛИЧКИ

Thanks for your attention!